

Submission Worksheet

IT202-450-M2026 / module-02-worksheet-php-challenge-scenarios

Submission Data

Course	IT202-450-M2026
Assignment	Module 02 Worksheet: PHP Challenge Scenarios
Student	Michos C. (mrc82)
Status	submitted
Worksheet Progress	100+%
Potential Grade	NaN/NaN (NaN%)
Started	6/16/2026, 12:02:50 AM
Updated	6/16/2026, 10:54:28 PM
Grading Link	https://courses.ethereallab.app/assignment/v3/IT202-450-M2026/module-02-worksheet-php-challenge-scenarios/grading/mrc82
View Link	https://courses.ethereallab.app/assignment/v3/IT202-450-M2026/module-02-worksheet-php-challenge-scenarios/mrc82

Instructions

Complete the Module 2 PHP challenge scenarios in the homework branch, then collect focused evidence for each scenario. It's best to solve and test the PHP files first, then fill out the worksheet; however, you may also collect evidence as each scenario is solved.

Assignment Setup

- Starter reference: [Module 02 Homework starter folder](#)
- Required branch: `Module02-Homework`
- Required homework folder: `public_html/m02/hw/`
- Required starting branch: `qa`
- Starter files to copy: `base.php`, `index.php`, `scenario1.php`, `scenario2.php`, `scenario3.php`, and `scenario4.php` (both `index.php` files should be included in their proper folders)
- Recommended order:
 - Read the whole worksheet
 - Copy the starter files; commit the baseline
 - Add planning comments, commit those comments
 - Solve each scenario, test through `qa`, then collect the worksheet evidence.

Git Workflow Checklist

1. Check out `qa`.
2. Pull the latest `qa` branch with `git pull origin qa`.
3. Create the homework branch with `git checkout -b Module02-Homework`.
4. Copy the starter files into the `m02/hw`.
5. Commit the baseline starter files before solving the scenarios.
6. Push the branch with `git push origin Module02-Homework`.
7. Open a pull request from `Module02-Homework` into `qa` and keep it open while you work.
8. Add planning comments before the final code for every scenario.

6. Push the branch with `git push origin Module02-Homework`.
7. Open a pull request from `Module02-Homework` into `qa` and keep it open while you work.
8. Add planning comments before the final code for every scenario.
9. Commit planning comments before the finished solution for each scenario.
10. Commit working code in small pieces as each scenario starts working.
11. After all scenarios pass locally, merge the homework pull request into `qa`.
12. Gather evidence from the deployed `qa` site and fill out the worksheet.
13. Export the worksheet PDF from the Learn Courses Platform.
14. Save the PDF in the repository under `docs/module02/`
15. Add, commit, and push the PDF to the `Module02-Homework` branch.

- `git add .`
- `git commit -m "Add module 2 worksheet PDF"`
- `git push origin Module02-Homework`

16. Upload the same PDF to Canvas.
17. Create and merge a pull request from `qa` into `prod` so the anticipated production URLs become live.
18. Switch back to `qa` with `git checkout qa`.
19. Pull the latest `qa` branch with `git pull origin qa`.

Shared Requirements

Apply these requirements to each scenario:

- Only edit code where the starter comments say edits are allowed.
- Add a comment with your UCID, date, and a brief summary of the solution approach.
- Add planning comments before the solution code.
- Commit the planning-comment stage before the final solution.
- Test the file through the supported server path before collecting output evidence. For deployed evidence, use the working `qa` URL after the site has deployed.
- Use your own code, output, branch, pull request, and deployed URLs for evidence.

Scenario Evidence Format

Each scenario uses the same evidence pattern:

- Images:
 - Show the relevant code and the matching browser output.
 - Make sure the UCID/date comment is visible in the code evidence.
 - One combined screenshot is acceptable if the code and output are both readable.
 - Use two screenshots when one image would be hard to read.
- URLs:
 - Provide the direct GitHub file link from the homework branch.
 - Provide the anticipated production URL for the PHP file.
 - Capture the working `qa` URL, change `-qa` to `-prod`, and keep the same path and filename.
 - Each file URL must end in `.php`; zero credit for that URL entry if it does not.
- Response:
 - Explain the logic in your own words.
 - Describe the steps or decisions your code makes.
 - Do not restate the prompt or list the requirements.

Submission Check

- Confirm the homework branch has been pushed.
- Confirm the pull request into `qa` exists.
- Confirm the baseline, planning, and solution commits are visible.

- Confirm the homework branch has been pushed.
- Confirm the pull request into `qa` exists.
- Confirm the baseline, planning, and solution commits are visible.
- Confirm the `qa` version was reviewed before promotion.
- Confirm the anticipated production URLs use the same path as the tested `qa` URLs with `-qa` changed to `-prod`.
- Confirm the worksheet PDF is committed under `docs/module02/` on the homework branch.
- Upload the same worksheet PDF to Canvas.

Submission Progress

100%

Section #1: (2 pts.) Scenario 1 Odds

100%

- File to edit and test: `public_html/m02/hw/scenario1.php`
- Goal: edit the `printodds()` function so it outputs only odd values from each array.
- Expected behavior: print odd values on one line, separated by commas and a space.
- Output restriction: there should be no leading comma and no trailing comma.
- Workflow:
 - Add/commit the baseline code
 - Add UCID/date and planning comments before the solution code.
 - Commit the planning comments before committing the finished solution.
 - Test the PHP file locally before collecting evidence on `qa`.

Task #1 (2 pts.) – Scenario 1 Evidence

100%

Combo #1

Part 1: Images 100%

Details

- Show the relevant code with the UCID/date comment visible.
- Show the full browser output after running the file.



Scenario 1 QA browser output showing only odd values.



Scenario 1 code with UCID/date planning comment and odd-values.

✓ Saved 6/16/2026 12:23:44 AM

Part 2: URLs 100%

Details

- Paste the direct GitHub file link from the homework branch.
 - The GitHub URL must end in `.php`; zero credit for this URL if it does not.
- Paste the anticipated production URL for the PHP file.
 - Capture the tested `qa` URL, change `-qa` to `-prod`, and keep the same path and filename.
 - The production URL must end in `.php`; zero credit for this URL if it does not.

URL #1

https://github.com/micopher/mrc82-IT202-450-M2026/blob/Module02-Homework/public_html/m02/hw/scenario1.php 🍏

URL #2

URL #1

https://github.com/micopher/mrc82-IT202-450-M2026/blob/Module02-Homework/public_html/m02/hw/scenario1.php 🍌

URL #2

<https://mrc82-it202-450-m2026-prod.onrender.com/m02/hw/scenario1.php> 🍌

✔ Saved 6/16/2026 12:23:44 AM

✎ Part 3: Response 100%

📄 Details

- Explain how your solution decides which values are odd.
- Explain how your solution avoids leading and trailing commas.
- Do not restate the prompt.

Response

My code makes an empty list first. Then it checks each number in the array one by one. If the number is odd, it adds it to the list. At the end, it uses implode with a comma and space to print the odd numbers neatly. This prevents an extra comma at the beginning or end.

Supports **bold**, *italic*, [links](url), ``code``, etc.

271 chars

✔ Saved 6/16/2026 12:23:44 AM

Section #2: (2 pts.) Scenario 2 Sum

100%

- File to edit and test: `public_html/m02/hw/scenario2.php`
- Goal: edit the `sumValues()` function so it sums the array values and formats the result with exactly two decimal places.
- Expected behavior: store the raw sum in `total`, then store the display-ready value in `modifiedTotal`.
- Output detail: values such as `0.1`, `1`, and `0.011` should display like `0.10`, `1.00`, and `0.01`.
- Workflow:
 - Add/commit the baseline code
 - Add UCID/date and planning comments before the solution code.
 - Commit the planning comments before committing the finished solution.
 - Test the PHP file locally before collecting evidence on `qa`.

📦 Task #1 (2 pts.) – Scenario 2 Evidence

100%

📦 Combo #1

🖼️ Part 1: Images 100%

📄 Details

- Show the relevant code with the UCID/date comment visible.
- Show the full browser output after running the file.



QA browser output showing summed values formatted to two decimals.



Code with UCID/date planning comment and sum formatting solution.

🔗 Part 2: URLs 100%

Part 2: URLs 100%

Details

- Paste the direct GitHub file link from the homework branch.
 - The GitHub URL must end in `.php`; zero credit for this URL if it does not.
- Paste the anticipated production URL for the PHP file.
 - Capture the tested `qa` URL, change `-qa` to `-prod`, and keep the same path and filename.
 - The production URL must end in `.php`; zero credit for this URL if it does not.

URL #1

https://github.com/micopher/mrc82-IT202-450-M2026/blob/Module02-Homework/public_html/m02/hw/scenario2.php 🍌

URL #2

<https://mrc82-it202-450-m2026-prod.onrender.com/m02/hw/scenario2.php> 🍌

Part 3: Response 100%

Details

- Explain how your solution sums the values.
- Explain where and how your code formats the final display value.
- Describe any floating-point math behavior you noticed.
- Do not restate the prompt.

Response

Code starts the total at 0, then it goes through each number in the array and adds it to the total. After the sum is done, it uses `number_format` to show the answer with exactly two decimal places. Some raw totals look slightly weird because I fear decimal math is not always perfect, but the formatted value makes the final output clean.

Supports **bold**, *italic*, `[links](url)`, ``code``, etc.

337 chars

Section #3: (2 pts.) Scenario 3 Conversion

100%

- File to edit and test: `public_html/m02/hw/scenario3.php`
- Goal: edit the `bePositive()` function so each value becomes positive, keeps or restores its original data type, and is stored in the correct slot of the `output` array.
- Expected behavior: each output value should be positive and should match the original value's intended type.
- The utility code will mark each value in the output noting its type such as `[I]`, `[D]`, `[S]`, etc. This isn't something you need to add/change, but it should be used to determine if you solved it correctly.
- Workflow:
 - Add/commit the baseline code
 - Add UCID/date and planning comments before the solution code.
 - Commit the planning comments before committing the finished solution.
 - Test the PHP file locally before collecting evidence on `qa`.

Task #1 (2 pts.) – Scenario 3 Evidence

100%

Combo #1

Part 1: Images 100%

Details

- Challenge 1: remove non-alphanumeric characters except spaces.
- Challenge 2: convert text to title case.
- Challenge 3: remove leading spaces, trailing spaces, and duplicate spaces between words.
- Note: Challenge 4 is extra credit item collected in another task.
- Expected behavior: assign the final phrase to `placeholderForModifiedPhrase`.
- Workflow:
 - Add/commit the baseline code
 - Add UCID/date and planning comments before the solution code.
 - Commit the planning comments before committing the finished solution.
 - Test the PHP file locally before collecting evidence on `qa`.

🔓 Task #1 (2 pts.) – Scenario 4 Evidence

100%

🔓 Combo #1

📁 Part 1: Images 100%

📄 Details

- Show the relevant code with the UCID/date comment visible.
- Show the full browser output after running the file.



QA browser output showing cleaned strings, title case, and middle characters.



Code with UCID/date planning comment and string transformation solution.

🔗 Part 2: URLs 100%

📄 Details

- Paste the direct GitHub file link from the homework branch.
 - The GitHub URL must end in `.php`; zero credit for this URL if it does not.
- Paste the anticipated production URL for the PHP file.
 - Capture the tested `qa` URL, change `-qa` to `-prod`, and keep the same path and filename.
 - The production URL must end in `.php`; zero credit for this URL if it does not.

URL #1

https://github.com/micopher/mrc82-IT202-450-M2026/blob/Module02-Homework/public_html/m02/hw/scenario4.php 🍏

URL #2

<https://mrc82-it202-450-m2026-prod.onrender.com/m02/hw/scenario4.php> 🍏

📝 Part 3: Response 100%

📄 Details

- Explain each string operation your solution uses.
- Explain the order of those operations.
- Explain why the order matters for the final phrase.
- Do not restate the prompt.

Response

This code first removes symbols and keeps only letters, numbers, and spaces. Then it trims the phrase and replaces extra spaces with one normal space. After that, it changes the phrase to title case. The order matters

This code first removes symbols and keeps only letters, numbers, and spaces. Then it trims the phrase and replaces extra spaces with one normal space. After that, it changes the phrase to title case. The order matters because the text should be cleaned first before making the final phrase. For the middle characters, the code removes the first and last character, then takes up to three characters from the middle.

Supports **bold**, *italic*, [links](url), `code`, etc.

416 chars

Section #5: (+1 pt.) Scenario 4 Extra Credit Middle Characters

100%

- File to edit and test: `public_html/m02/hw/scenario4.php`
- Goal: add the extra credit middle-character result to `transformText`.
- Challenge 4: extract up to the middle three characters from each cleaned phrase and assign the result to `placeholderForMiddleCharacters`.
- **Hint:** use the middle character +/- 1 (excluding the first and last characters) to find up to the middle 3 characters.
 - Examples: `short` should produce `hor`, `even` should produce `ve`, `eve` should produce `v`, and `abcdef` should produce `cde`.
- **Not enough characters rule:** assign `Not enough characters` when no middle characters remain after excluding the first and last character. For example, `a` should produce `Not enough characters`.
- URL note: this is part of Scenario 4, so it does not need a separate URL entry.
- Workflow:
 - Add UCID/date and planning comments before the solution code.
 - Commit the planning comments before committing the finished solution.
 - Test the PHP file locally before collecting evidence on `qa`.

Task #1 (+ 1 pt.) – Extra Credit Evidence

100%

Details

- Collect evidence for the extra credit solution.

Combo #1

Part 1: Images 100%

Details

- Show the relevant code with the UCID/date comment visible.
- Show the full browser output after running the file.



Extra credit browser output showing middle characters for each phrase.



Extra credit code showing the middle character logic.

Part 2: Response 100%

Details

- Explain how your solution finds the middle of the string.
- Explain how your solution handles strings that are too short.
- Explain how your solution handles the different even/odd length phrases.
- Do not restate the prompt.

- Explain how your solution handles the different even/odd length phrases.
- Do not restate the prompt.

Response

Completed the extra credit as part of Scenario 4. So the code uses the cleaned final phrase, removes the first and last character, and then takes up to three characters from the middle. If the phrase is too short, it shows "Not enough characters.". This is why the same Scenario 4 output also shows the extra credit middle character results.

Supports **bold**, *italic*, [links](url), ``code``, etc.

342 chars

Section #6: (2 pts.) Misc

67%

Task #1 (0.67 pts.) – GitHub Details

100%

Details

- Show the repository evidence for the Module 02 homework.

Combo #1

Part 1: Images 100%

Details

- Screenshot the Commits tab of the pull request.
- The screenshot should show at least the baseline commit, planning-comment commits, and solution commits.



Module 2 pull request commits showing baseline, planning, and solution commits.

Part 2: URLs 100%

Details

- Paste the pull request URL.
- The URL must end in `/pull/#`, where `#` is the pull request number; zero credit for this URL if it does not.

URL #1

<https://github.com/micopher/mrc82-IT202-450-M2026/pull/5> 🍎

Task #2 (0.67 pts.) – WakaTime Activity

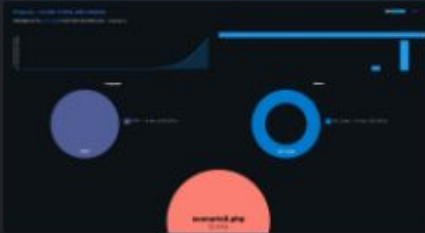
100%

Task #2 (0.67 pts.) WakaTime Activity

100%

Details

- Visit the WakaTime dashboard.
- Open **Projects**.
- Find your repository.
- Screenshot the overall time near the repository name (top of page).
- Screenshot the file-level activity (near the bottom).
- The purpose is evidence of activity; the duration isn't tied to the grade.
- The visual graphs are not required.



Time project dashboard showing total time for my IT202 repo.



Time file activity showing Module 2 PHP files worked on.

Task #3 (0.67 pts.) – Reflection

Details

- Answer each question with at least a few reasonable sentences.

Task #1 (0.22 pts.) – What did you learn during this assignment?

100%

Response

I learned how to use PHP, Git, GitHub pull requests, and Render together. I also learned how to test my work locally before pushing it.

Supports ****bold****, **italic**, [links](url), ``code``, etc.

136 chars

Task #2 (0.22 pts.) – What was the easiest part of the assignment?

100%

Response

The easiest part was checking the browser output, also working on the code. The page made it clear if the result was correct.

Supports ****bold****, **italic**, [links](url), ``code``, etc.

126 chars

Task #3 (0.22 pts.) – What was the hardest part of the assignment?

100%

Response

Response

The hardest part was keeping track of the branches, QA, prod, and Render. It got confusing, but I, hopefully, understand it better now. Also reading the first Scenario was a little hard too, alot to understand before proceeding with it.

Supports **bold**, *italic*, [links](url), ``code``, etc.

237 chars

⌂ Saving...

End of Assignment - submission has been recorded.